

ELE216

Développement de logiciels en génie électrique

Pascal Giard

Département de génie électrique

Cours 2:

UML, Encapsulation, Revue de code, et Tests

COURS 2

UML: diagramme de classe

Principes de conception: encapsulation

Revue de code

Tests unitaires

UML: diagramme de classe

UML

- ❑ UML = *Unified Modeling Language*.
- ❑ UML est un langage graphique servant à décrire l'architecture et le fonctionnement d'un logiciel
- ❑ On s'en sert à l'étape de conception pour décrire l'architecture du logiciel. (avant de coder!!!)
- ❑ Plusieurs types de diagramme, par exemple:
 - ❖ Diagramme de classes: décrit l'architecture, les relations entre les modules
 - ❖ Diagramme de séquence: décrit le comportement du logiciel
 - ❖ Diagramme d'états: décrit aussi le comportement du logiciel

Une image vaut mille mots

Diagramme de classes



Diagramme de séquence

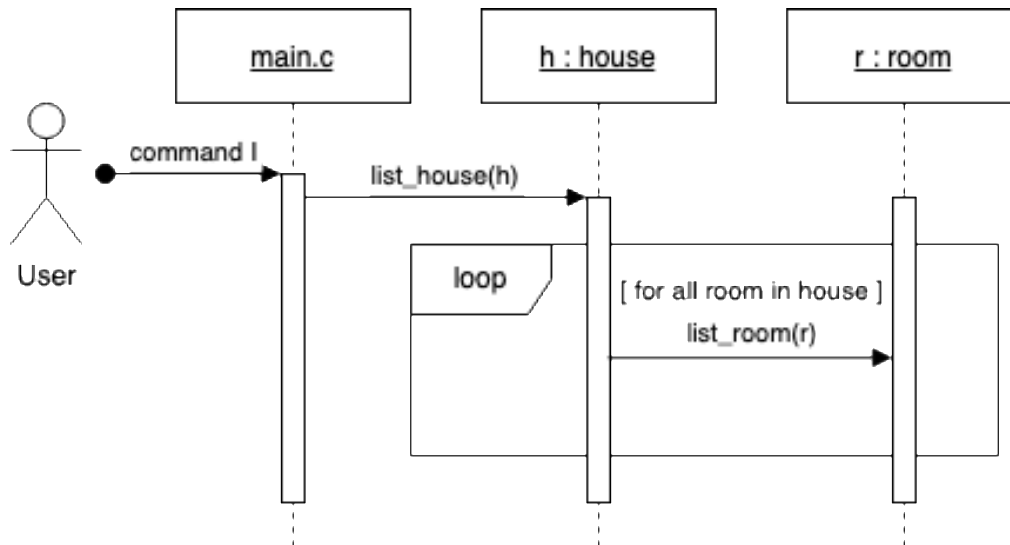


Diagramme d'états

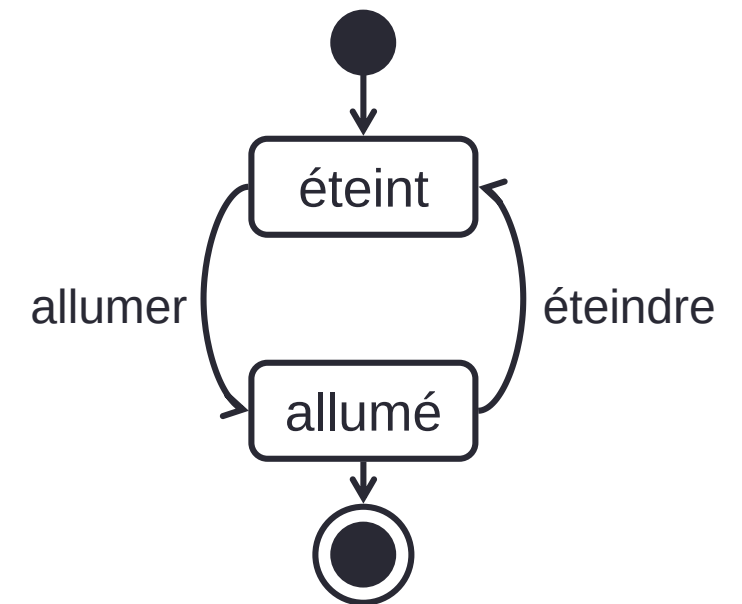


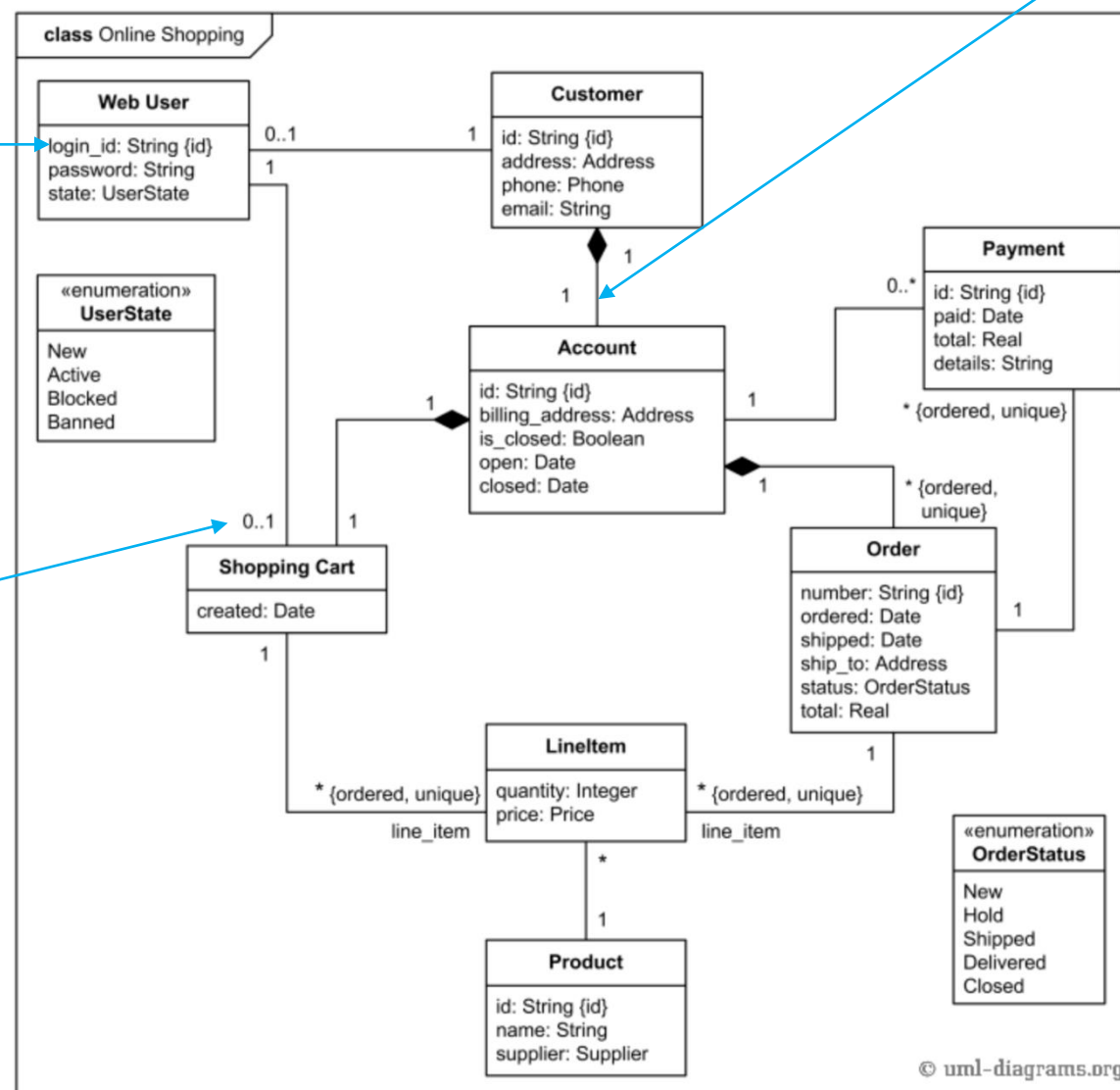
Diagramme de classes

Que veut dire ce connecteur?

En pratique, à quoi pourrait correspondre chaque boîte dans un programme en C?

Que veulent dire les nombres à chaque extrémité des connecteurs?

À quoi sert ce type de diagramme?



Exemple: le zoo

Fonctions Attributs

Animal	
	- nom : string - id : int + age : int + masse : double
	+ setNom(nouvNom : string) : void + getNom() : string + manger() : void + avancer(dist : int) : void



Visibilité:

- Privé (-) : utilisé localement, en C, n'est pas défini dans le fichier de spécification .h
- Public (+): en C, défini dans le fichier .h, accessible de l'externe

Différents niveaux de détail possibles

Non documenté

Voiture

Documenté

Voiture
marque immatriculation propriétaire
démarrer conduire arrêter

Détaillé

Voiture
+ marque : String + Immatriculation : String + couleur : String - poidsVide : entier - dateFabrication : Date - <u>propriétaire</u> : Personne
+ démarrer() + arrêter() + conduire(a : Lieu, b : Lieu) - vendre(prix : entier long)

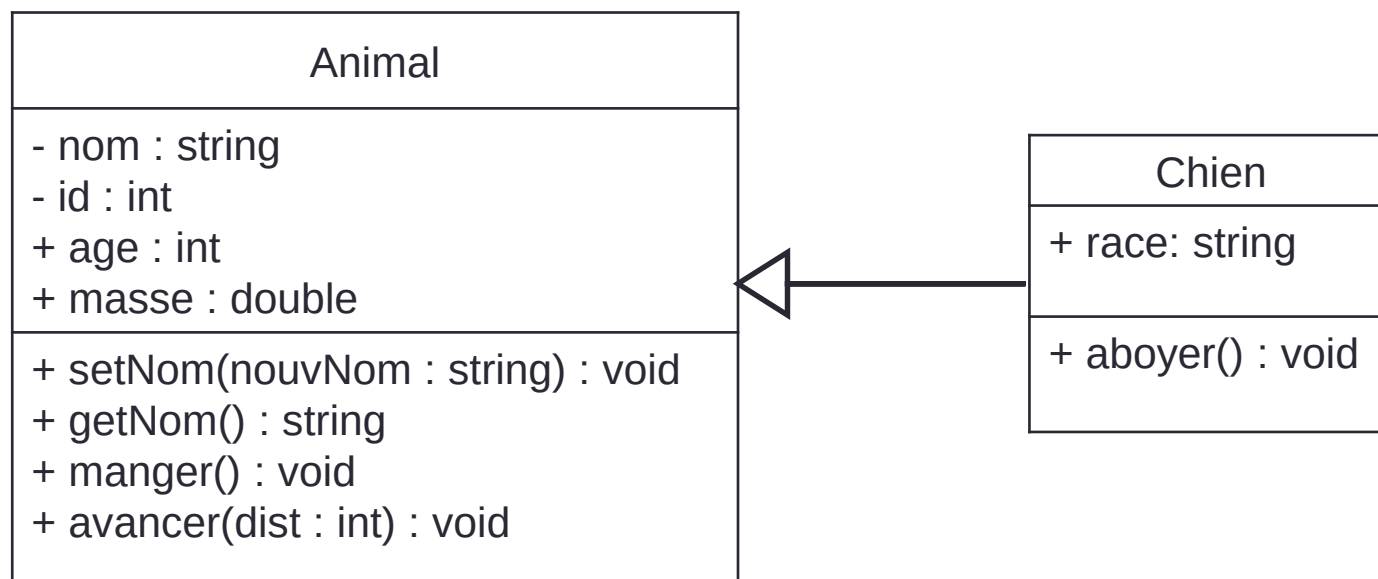
Remarques:

- Souligné veut dire statique
- On pourrait fournir encore plus de détails, mais ça dépasse le cadre du cours

Relations entre classes ou structures

1. L'héritage →

❖ Possible d'implémenter l'héritage en C mais nous n'irons pas là



Relations entre classes ou structures

1. L'héritage

❖ Possible de faire de l'orienté objet en C mais nous n'irons pas là

2. L'association



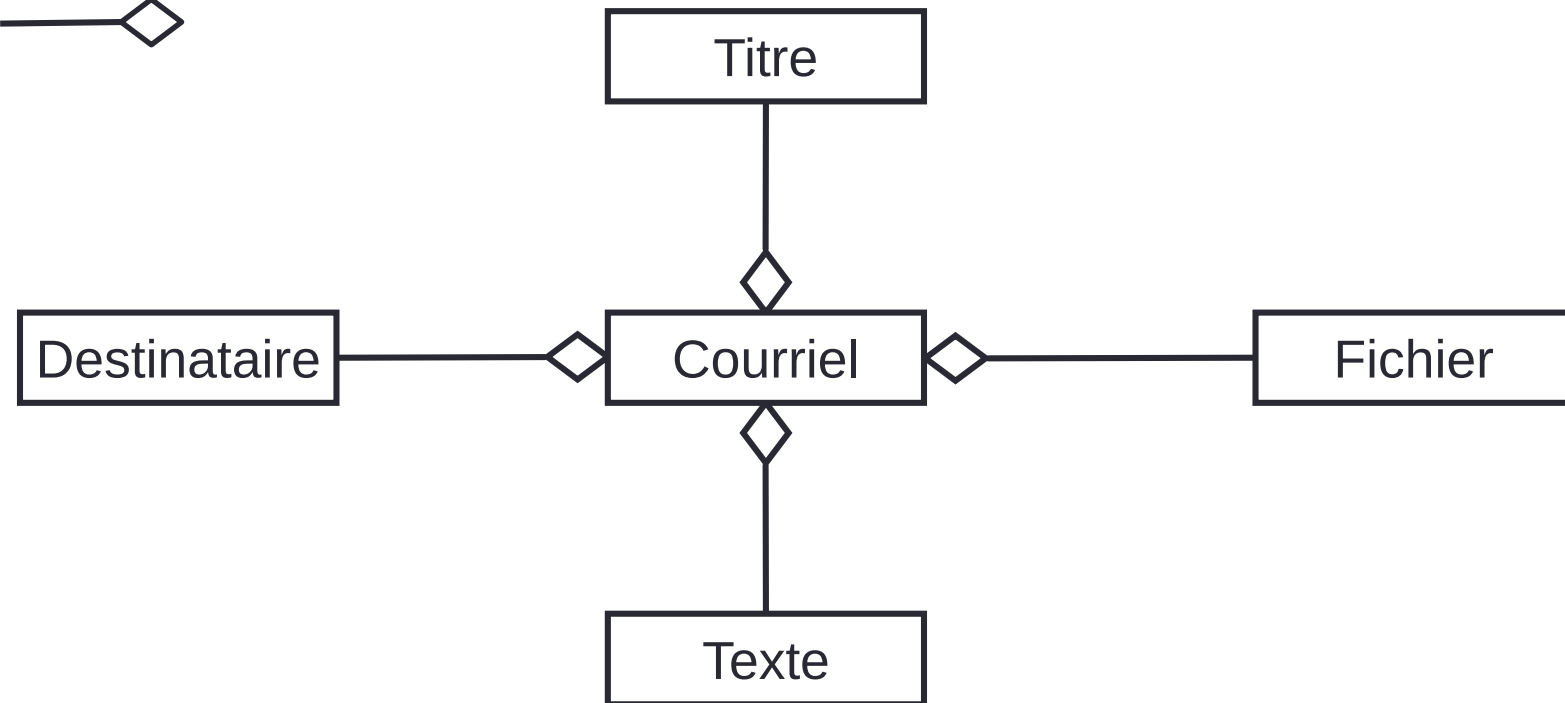
Relations entre classes ou structures

1. L'héritage

❖ Possible de faire de l'orienté objet en C mais nous n'irons pas là

2. L'association

3. L'agrégation



Relations entre classes ou structures

1. L'héritage 

❖ Possible de faire de l'orienté objet en C mais nous n'irons pas là

2. L'association 

3. L'agrégation 

4. La composition 



Exercice

- ☐ Faites un diagramme de classes pour un zoo contenant des loups et des girafes
- ☐ Il y a au moins un loup et une girafe
- ☐ Le zoo contient au moins une meute de loups
- ☐ La meute de loups n'inclut pas nécessairement tous les loups
- ☐ Toutes les femelles girafes sont regroupées dans une crèche (sorte de groupe de femelles girafes)
- ☐ La crèche n'admet pas de girafe mâle
- ☐ Les loups hurlent et les girafes meuglent

Principe de conception : encapsulation

Principes de conception du logiciel

Les principes suivants doivent être appliqués pour faciliter *l'évolution et la réutilisation* du code:

❑ Encapsulation

- ❖ Concevoir le code en modules indépendants les uns des autres.
- ❖ Le module doit cacher sa mécanique interne, et offrir des fonctions sécuritaires.

❑ Réduction du couplage

- ❖ Réduire le plus possible les appels de fonction entre les différents modules
- ❖ Éviter d'utiliser des variables globales

❑ Amélioration de la cohésion du code

- ❖ Éviter la duplication de code ou de variables

❑ Suivre une norme de programmation



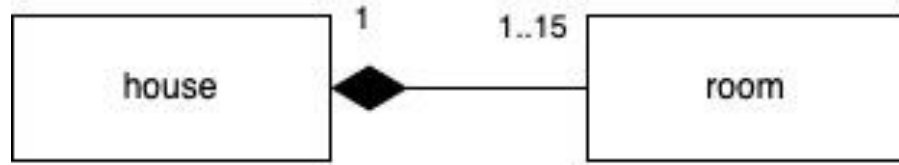
Très important
pour ELE216
tout le temps

Encapsulation

- ❑ Comment concevoir un module bien encapsulé en C?
 - ❖ Les données du module doivent être cachées du programme appelant.
 - Elles sont accessibles seulement par les fonctions publiques du module via un pointeur « aveugle ».
 - Les structures des données sont définies dans le fichier « .c »
 - ❖ Les fonctions publiques du module sont prototypées dans le fichier « .h »
 - ❖ Aucune référence croisée
 - Par exemple: une fonction du module 1 qui appelle une fonction du module 2 et une autre fonction du module 2 qui appelle une fonction du module 1

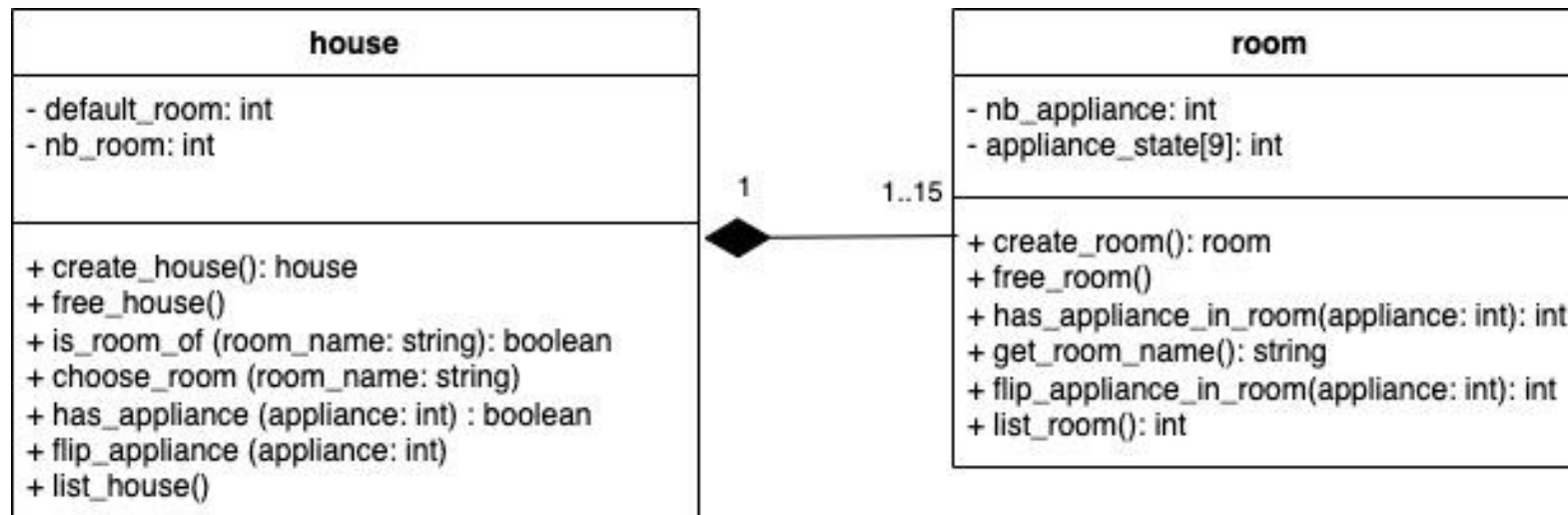
Encapsulation: exemple

- ❑ Diagramme de classes – version simplifiée



Encapsulation: exemple

❑ Diagramme de classes – version détaillée



Encapsulation: exemple

Code C

```

1  /*
2   * house.h
3   *
4   * Created on: Sep. 11, 2021
5   * Author: vincent
6   */
7
8  #ifndef HDR_HOUSE_H_
9  #define HDR_HOUSE_H_
10
11  typedef struct house house_t;
12
13  house_t* create_house();
14  void free_house(house_t*);
15  int is_room_of(house_t*, char* );
16  void choose_room(house_t*, char*);
17  int has_appliance(house_t*, int);
18  void flip_appliance(house_t*, int);
19  void list_house(house_t*);
20
21  #endif /* HDR_HOUSE_H_ */
22

```

Pointeur
aveugle

```

1  /*
2   * house.c
3   *
4   * Created on: Sep. 11, 2021
5   * Author: vincent
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <string.h>
11 #include "house.h"
12 #include "room.h"
13
14 #define MAX_ROOMS 15
15
16 struct house {
17     int default_room;
18     int nb_rooms;
19     room_t* rooms[MAX_ROOMS];
20 };
21
22 // private functions prototypes
23 int get_room_index(house_t*, char*);
24 room_t* get_default_room(house_t*);
25 char* get_default_room_name(house_t*);
26
27 // allocate memory for a house, and create some rooms
28 house_t* create_house() {
29     house_t* house;
30     int i;
31
32     house = (house_t*) malloc(sizeof(house_t));
33     if (house == NULL) return NULL;
34     house->default_room = 0;
35
36

```

Revue de code

Exemple: programme « contrôle_maison »

Parmi les stratégies de vérifications

□ Statique

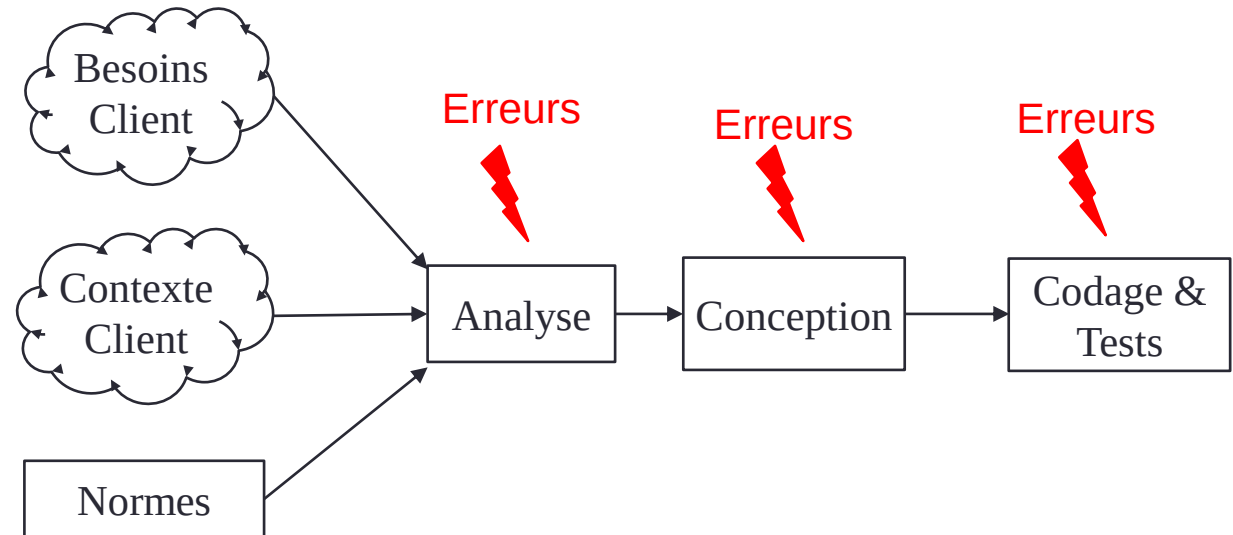
- ❖ Revue de code

□ Dynamiques

- ❖ Tests de boîte noire
- ❖ Tests de boîte blanche

Revue de code - constat

- ❑ Une personne est faible pour trouver ses propres erreurs.
- ❑ L'erreur peut être due à une mauvaise compréhension du problème à résoudre.
- ❑ Si une erreur se produit durant l'analyse ou la conception.
Comment les trouver?



Revue de code

- ❑ Revue du code et de son architecture effectuée avec un pair
- ❑ Avantages:
 - ❖ Peut se faire tôt dans le processus de développement
 - ❖ Peu coûteux

Revue de code - méthodologie

- ❑ Impliquer une (ou plusieurs) autre personne qui n'a *pas de connaissance à priori* sur le produit.
- ❑ Le programmeur revoit le code pas à pas, et explique à l'autre personne son raisonnement pour résoudre le problème.
- ❑ Toute la documentation (cas d'utilisation, modèle, code, documentation) doit être disponible.
- ❑ Les fautes doivent être consignées par écrit.

Revue de code – exemple/exercice

- ❑ Revue du code du programme « contrôle_maison »
- ❑ Récupérez le code sur Gitlab (voir le lien fourni sur Moodle)

Revue de code de contrôle_maison – Parmi les choses à regarder

❑ Organisation:

- ❖ Est-ce que les déclarations (publiques) sont dans des fichiers d'entête?
- ❖ Est-ce que les inclusions dans les fichiers d'entête sont protégées? (`#ifndef`, `#endif`)
- ❖ Est-ce qu'il y a des commentaires pertinents décrivant la fonctionnalité?

❑ Structure: Est-ce que l'implémentation correspond à ce qui a été décrit dans le diagramme de classe? (voir *slide 18*)

❑ Gestion de la mémoire: Est-ce que chaque allocation mémoire (*malloc*) a un équivalent pour la libération (*free*)?

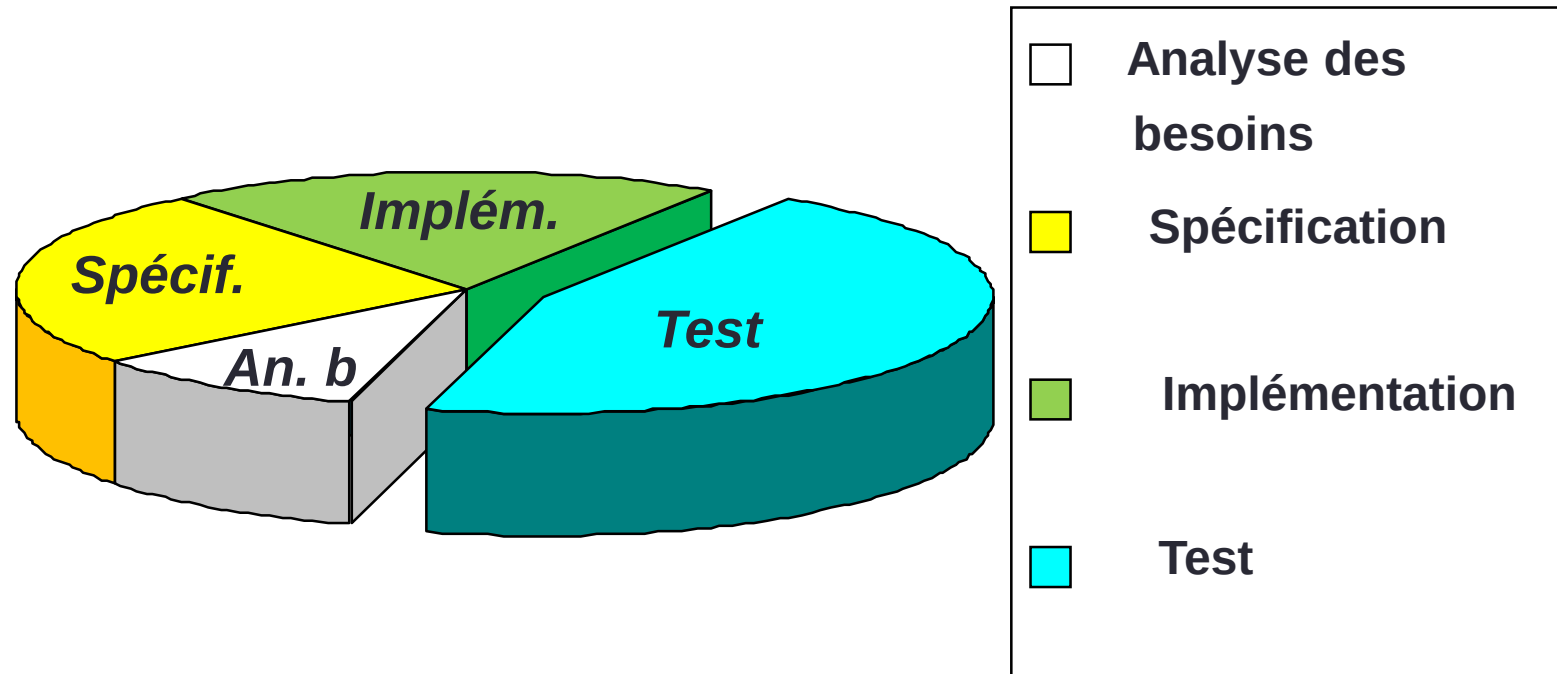
❑ Validation des entrées: Est-ce que ce qui est fourni par la personne utilisatrice de l'application est validé avant d'être utilisé?

Remarque: Ce n'est vraiment pas une liste exhaustive...

Tests (unitaires)

L'importance des tests

Le coût du test dans le développement



+ maintenance = 80 % du coût global de développement !!!

Des défaillances qui sont passées à l'histoire

❑ Le vidéoclip qui a brisé Youtube



PSY - GANGNAM STYLE(강남스타일) M/V

3.8B views • 8 years ago



PSY - 'I LUV IT' M/V @ <https://youtu.be/Xvjnoagk6GU> PSY - 'New Face' M/V @<https://youtu.be/OwJPPaEyqhl> PSY - 8TH ALBUM ...

CC

❑ Raison: débordement de tampon (“overflow”)

- ❖ Jusque là, Youtube utilisait un entier de 32 bits pour compter le nombre de visionnements. Cette chanson a été la première à dépasser la limite alors permise de 2 147 483 647 visionnements.

Des défaillances qui sont passées à l'histoire

- ❑ Explosion de la fusée Ariane 5 (4 juin 1996)
 - ❖ Une décennie et 8 milliards de dollars US en développement
 - ❖ Explosion et perte de 370 millions de dollars US 40 secondes après le décollage

- ❑ Raison: débordement de tampon ("overflow")
 - ❖ Tentative d'insertion d'un nombre entier sur 64 bits dans un tampon de 16 bits

Des défaillances qui sont passées à l'histoire

- ❑ Destruction du Mars Climate Orbiter de la NASA (1998)
 - ❖ Projet de 125 millions de dollars US
 - ❖ La sonde devait recueillir des données sur le climat de la planète Mars
 - ❖ Après 7 mois à voyager jusqu'à Mars, la sonde tente de se placer sur l'orbite prévue mais finit trop près de Mars et est détruite en traversant à grande vitesse la partie supérieure de son atmosphère.

- ❑ Raison: la composante du logiciel produite par Lockheed Martin, un sous-traitant de la NASA, produisait des données en unités impériales au lieu des unités SI prévues par le contrat.

Tester, qu'est-ce que ça veut dire?

Essayer pour voir si ça marche.

Tester, qu'est-ce que ça veut dire?

Essayer pour voir si ça marche.

- Pourquoi essayer?
- Quand essayer?
- Comment essayer?
- Qui doit essayer?

Tester, qu'est-ce que ça veut dire?

Essayer pour voir si ça marche.

- Pourquoi essayer?
- Quand essayer?
- Comment essayer?
- Qui doit essayer?

- Qu'est-ce qui devrait marcher?
- Ça veut dire quoi, marcher?
- Si ça marche aujourd'hui, est-ce que ça va marcher demain?

Tester, qu'est-ce que ça veut dire?

Essayer pour voir si ça marche.

- Pourquoi essayer?
- Quand essayer?
- Comment essayer?
- Qui doit essayer?
- Qu'y a-t-il à voir?
- Que faut-il regarder?
- Est-ce que tout est visible?
- Qu'est-ce qu'on cherche?
- Comment le regarder?
- Qu'est-ce qui devrait marcher?
- Ça veut dire quoi, marcher?
- Si ça marche aujourd'hui, est-ce que ça va marcher demain?

Stratégies de tests

❖ Tests de boîte noire

- On teste le programme sans connaître le code
- On se sert des spécifications pour connaître le comportement attendu.

❖ Tests de boîte blanche

- On teste le programme en examinant le code.
- On veut exercer toutes les lignes de code
 - Essayer les conditions vraie et fausse des « if »
 - Rouler les boucles: 0 fois, 1 fois et N fois.

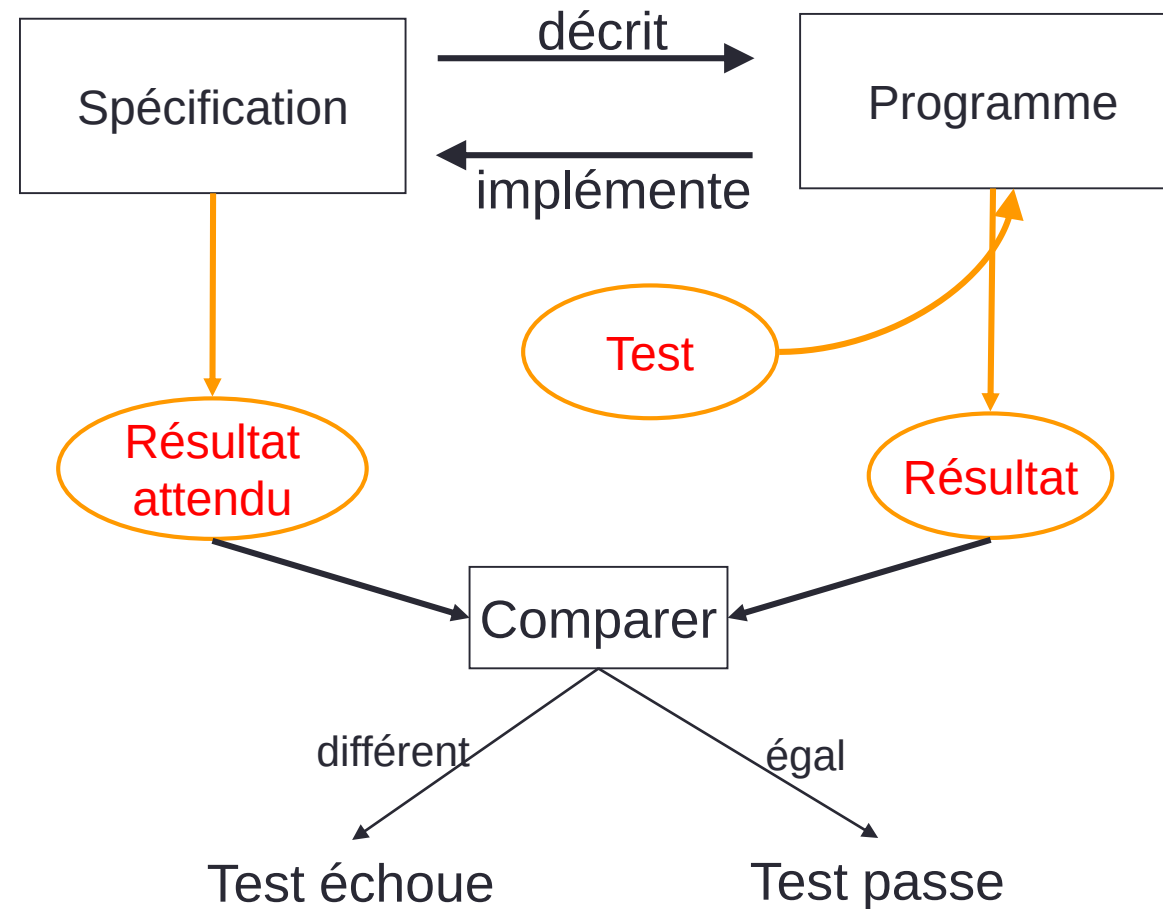
Quoi tester

- ❑ Une fonction
- ❑ Un module
- ❑ Une application
- ❑ Un système
 - ❖ Ses fonctions
 - ❖ Sa performance
 - ❖ Sa robustesse
 - ❖ etc.

Avec quoi tester

- ❑ Une spécification qui décrit ce qui est attendu
 - ❖ un cahier des charges (en langue naturelle)
 - ❖ commentaires dans le code
 - ❖ le nom des fonctions (par exemple dans un fichier .h)
 - ❖ un modèle UML
 - ❖ etc.

Test de logiciel



Les niveaux de test

❑ Unitaire

- ❖ début du développement, sur les modules
- ❖ teste des petits bouts de code

❑ Intégration

- ❖ certains modules sont terminés
- ❖ teste l'interaction entre les modules

❑ Validation

- ❖ quand tous les modules sont prêts

❑ Système

- ❖ logiciel et matériel, performance, robustesse, ergonomie, déploiement, etc.

Tests unitaires

- ❑ Pour tester intensivement un module précis et petit, parfois juste une fonction
- ❑ Validation d'un module indépendamment des autres
- ❑ Stratégie boîte noire et/ou boîte blanche
- ❑ Généralement automatisés
- ❑ Les tests unitaires sont idéalement refaits chaque fois qu'une modification est apportée (même si cette modification est dans un autre module/une autre fonction). Cela s'appelle alors des *tests de régression*.

Les tests unitaires

- ❑ On développe un « jeu de tests » qui est un ensemble de cas de tests.
- ❑ On écrit le jeu de test en C dans un répertoire séparé.
- ❑ Toutes les fonctions d'un module doivent être exercées.
- ❑ Les cas des tests doivent exercer les situations *valides* et les situations *invalides*.
- ❑ On essaie de couvrir, dans la mesure du possible, toutes les branches du code source (tests de boîte blanche).
- ❑ Les résultats de tests doivent être documentés.

Tests unitaires - exemple

- ❑ programme « controle_maison »
- ❑ Regardez les tests dans le programme test/test_room.c
- ❑ Exécutez le programme.
- ❑ Les tests sont-ils exhaustifs?
- ❑ Couvrent-ils l'ensemble des branches du code présent dans room.c?

Tests unitaires - exercice

- ☐ Module « fraction »
- ☐ Concevez un jeu de tests permettant de valider (ou d'invalidier, préférablement) le fonctionnement du module
 - ☐ Ajouts au main.
 - ☐ Tests à boîte noire (basés sur uniquement les spécifications attendues ou l'information présente dans le .h)
 - ☐ N'oubliez pas d'inclure des cas valides et invalides
 - ☐ Tests à boîte blanche (ajoutez des cas de test pour mieux couvrir l'ensemble des branches du code source dans le .c)